Implementation and Enhancement of BLUE Active Queue Management in NS3

Network Simulation Project Proposal

Shatabdi Dutta Chowdhury
Student ID: 2105123

Department of Computer Science & Engineering

January 28, 2026

Problem Definition & Challenges

Core Problem

Internet Congestion: Exponential traffic growth causes high packet loss rates and potential congestion collapse

Key Challenges:

- High packet loss rates (10-30%)
- Poor link utilization
- Unfair bandwidth distribution
- Large buffer requirements

Impact:

- Wasted network resources
- Increased end-to-end delay
- Poor QoS for applications
- TCP throughput degradation

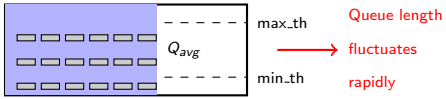
Question

How can we efficiently manage congestion with minimal packet loss and buffer space?

Existing System: RED Algorithm Problems

Random Early Detection (RED):

- Uses queue length as congestion indicator
- Maintains exponentially-weighted moving average
- Drops/marks packets when Q_{avg} exceeds thresholds



RED Queue

Performance Issues:

- 15-20% packet loss (high load)
- Oscillating queue behavior
- Link underutilization periods

Fundamental Issues

Fundamental Issues

Problem 1: Queue length $\neq$ Congestion severity
(1 aggressive flow vs 1000 flows looks same)

Problem 2: Requires large buffers ($2\times$ BDP)
(Increases delay, impractical for legacy routers)

Problem 3: Parameter sensitivity
(min_th, max_th, max_p, w_q must be tuned)

Proposed Solution: BLUE Algorithm

Key Innovation

Use **packet loss** and **link utilization** instead of queue length for congestion management

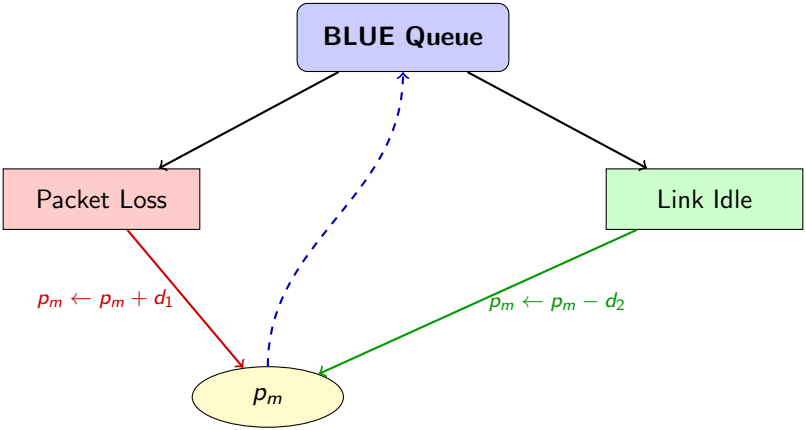
Core Concept:

- Single marking probability: p_m
- Packet loss event $\rightarrow$ p_m increases
- Link idle event $\rightarrow$ p_m decreases
- "Learns" optimal marking rate

BLUE Algorithm : Advantages

- ✓ Works with small buffers
- ✓ Low packet loss
- ✓ High link utilization
- ✓ Simple & efficient
- ✓ No complex tuning

BLUE Feedback Loop



BLUE Algorithm: Pseudocode

Algorithm 1 BLUE Algorithm (Part 1)

```
1:  Initialize:
2:  p_m ← 0
3:  last_update ← 0
4:  Parameters: d_1, d_2, freeze_time
5:
6:  function OnPacketLoss
7:      if (now − last_update) > freeze_time then
8:          p_m ← p_m + d_1
9:          p_m ← min(p_m, 1.0)
10:         last_update ← now
11:     end if
12: end function
```

Algorithm 2 BLUE Algorithm (Part 2)

```
1:  function OnLinkIdle
2:      if (now − last_update) > freeze_time then
3:          p_m ← p_m − d_2
4:          p_m ← max(p_m, 0.0)
5:          last_update ← now
6:      end if
7:  end function
8:
9:  function OnPacketArrival(packet)
10:     rand ← Random[0, 1)
11:     if rand < p_m then
12:         Mark / Drop packet
13:     else
14:         Enqueue packet
15:     end if
```

Why BLUE works

Adaptive Learning

p_m converges to optimal value that:

- Prevents packet loss
- Maintains high utilization
- Avoids queue overflow

Comparison with RED:

Metric	RED	BLUE
Packet Loss	15-20%	¡1%
Buffer Needed	2×BDP	0.5×BDP
Parameters	4	3
Complexity	High	Low

Simulation Network Topology

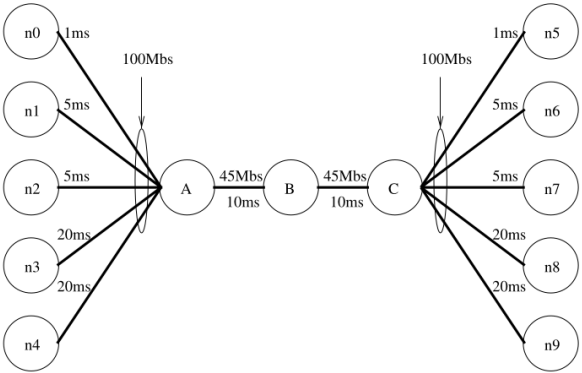


Figure: Network Topology

Configurations: RED vs BLUE

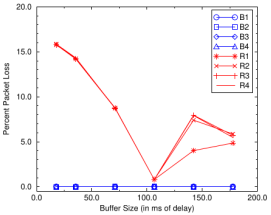
Configuration	w_q
$R1$	0.0002
$R2$	0.002
$R3$	0.02
$R4$	0.2

Table 1: RED configurations

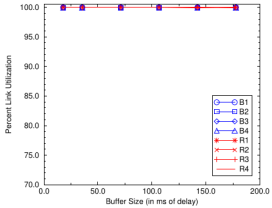
Configuration	$freeze\_time$	$d1$	$d2$
$B1$	$10ms$	0.0025	0.00025
$B2$	$100ms$	0.0025	0.00025
$B3$	$10ms$	0.02	0.002
$B4$	$100ms$	0.02	0.002

Table 2: BLUE configurations

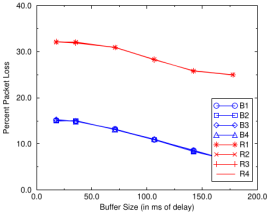
Expected Performance: RED vs BLUE



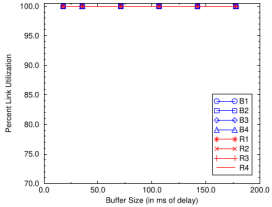
(a) Loss Rates (1000 sources)



(b) Link Utilization (1000 sources)



(c) Loss Rates (4000 sources)



(d) Link Utilization (4000 sources)

Proposed Modification 1: Adaptive Freeze Time

Our Contribution (Implemented)

This work implements an adaptive freeze_time mechanism to improve BLUE responsiveness in dynamic traffic conditions.

Problem

- Standard BLUE uses fixed freeze_time
- Slow reaction to sudden traffic changes
- Suboptimal performance in dynamic networks

Proposed Modification 1: Solution

Proposed Solution

Freeze time is dynamically adjusted based on congestion level:

$$freeze\_time = f(loss\_rate)$$

- High congestion $\rightarrow$ smaller freeze_time (faster updates)
- Low congestion $\rightarrow$ larger freeze_time (better stability)

Adaptive Rule (Used in Our Implementation)

$$freeze\_time = \begin{cases} 10\ ms & \text{if } loss > 5\% \\ 50\ ms & \text{if } 1\% < loss \leq 5\% \\ 100\ ms & \text{if } loss \leq 1\% \end{cases}$$

Proposed Modification 2: Loss Pattern Analysis

Proposed Idea (Not Implemented)

This modification is proposed as future work and is not implemented in the current study.

Motivation

- BLUE treats all packet losses equally
- No distinction between bursty and sustained losses

Proposed Modification 2: Proposed Approach

Proposed Approach

- Analyze loss patterns over time
- Adjust increment parameter d_1 accordingly

Expected Benefit

More precise congestion response with fewer unnecessary drops

Proposed Modification 3: Hybrid BLUE–RED

Proposed Idea (Not Implemented)

This hybrid approach is suggested as future work and is not implemented in this project.

Motivation

- BLUE ignores queue length information
- Sudden burst congestion may go undetected

Proposed Modification 3 : Hybrid Model

Proposed Hybrid Model

Combine BLUE and RED decisions:

$$p_{final} = \alpha \cdot p_m^{BLUE} + (1 - \alpha) \cdot p^{RED}$$

- $\alpha = 0.8$ (BLUE dominant)
- RED acts as a safety mechanism
- RED triggered when queue exceeds 80%

Expected Benefit:

- Improved burst handling
- Balanced delay and throughput

Implementation Plan

But I will implement modification-1 at first. If I get enough time, then I may implement modification-2 or modification-3.

Research-paper Link:

View the Paper

Thank you